

✓ Horizon Tech Internship – Task 3

Sales Data Analysis & Exploratory Data Analysis

Objective

The objective of this project is to analyze sales data, clean and preprocess the dataset, perform exploratory data analysis, identify trends and patterns, and generate meaningful business insights using Python.

Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Google Colab

Start coding or [generate](#) with AI.

```
from google.colab import files

uploaded = files.upload()
```

[Choose files](#) No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving Data Set- Inc5000 Company List_2014.csv to Data Set- Inc5000 Company List_2014 (1

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('Data Set- Inc5000 Company List_2014.csv')

print("Dataset loaded successfully!")
```

Dataset loaded successfully!

```
df.head()
```

	_input	_num	_widgetName	_source	_resultNumber	_page
0	NaN	1	Inc5000 Company List	Inc5000 Company List	1	http://www.inc.com/inc5000list/json/inc5000_
1	NaN	2	Inc5000 Company List	Inc5000 Company List	2	http://www.inc.com/inc5000list/json/inc5000_
2	NaN	3	Inc5000 Company List	Inc5000 Company List	3	http://www.inc.com/inc5000list/json/inc5000_
3	NaN	4	Inc5000 Company List	Inc5000 Company List	4	http://www.inc.com/inc5000list/json/inc5000_
4	NaN	5	Inc5000 Company List	Inc5000 Company List	5	http://www.inc.com/inc5000list/json/inc5000_

```
df.shape
```

```
(5000, 19)
```

```
df.columns
```

```
Index(['_input', '_num', '_widgetName', '_source', '_resultNumber', '_pageUrl',
      'id', 'rank', 'workers', 'company', 'url', 'state_l', 'state_s', 'city',
      'metro', 'growth', 'revenue', 'industry', 'yrs_on_list'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   _input                 0 non-null     float64
1   _num                  5000 non-null  int64
2   _widgetName           5000 non-null  object
3   _source               5000 non-null  object
4   _resultNumber         5000 non-null  int64
5   _pageUrl              5000 non-null  object
6   id                    5000 non-null  int64
7   rank                  5000 non-null  int64
8   workers               5000 non-null  int64
9   company               5000 non-null  object
10  url                   5000 non-null  object
11  state_l               5000 non-null  object
12  state_s               5000 non-null  object
13  city                  5000 non-null  object
14  metro                 4942 non-null  object
15  growth                5000 non-null  float64
16  revenue               5000 non-null  int64
17  industry              5000 non-null  object
18  yrs_on_list           5000 non-null  int64
dtypes: float64(2), int64(7), object(10)
memory usage: 742.3+ KB
```

df.describe()

	_input	_num	_resultNumber	id	rank	workers	growt
count	0.0	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.00000
mean	NaN	2500.500000	2500.500000	20036.575400	2500.500000	208.969800	516.43989
std	NaN	1443.520003	1443.520003	8041.042917	1443.520003	1074.600125	2786.06179
min	NaN	1.000000	1.000000	4.000000	1.000000	0.000000	42.44700
25%	NaN	1250.750000	1250.750000	19575.250000	1250.750000	24.000000	84.20825
50%	NaN	2500.500000	2500.500000	23291.500000	2500.500000	50.000000	151.72280
75%	NaN	3750.250000	3750.250000	25370.250000	3750.250000	125.000000	347.65282
max	NaN	5000.000000	5000.000000	26620.000000	5000.000000	34219.000000	158956.91060

df.isnull().sum()

	0
_input	5000
_num	0
_widgetName	0
_source	0
_resultNumber	0
_pageUrl	0
id	0
rank	0
workers	0
company	0
url	0
state_l	0
state_s	0
city	0
metro	58
growth	0
revenue	0
industry	0
yrs_on_list	0

dtype: int64

```
# Remove unnecessary technical columns

columns_to_remove = [
    '_input',
    '_widgetName',
    '_source',
    '_resultNumber',
    '_pageUrl'
]

df = df.drop(columns=columns_to_remove, errors='ignore')

print("Unnecessary columns removed.")
```

Unnecessary columns removed.

```
df.columns
```

```
Index(['_num', 'id', 'rank', 'workers', 'company', 'url', 'state_l', 'state_s',
      'city', 'metro', 'growth', 'revenue', 'industry', 'yrs_on_list'],
      dtype='object')
```

```
df['metro'] = df['metro'].fillna('Unknown')

print("Missing metro values handled.")
```

Missing metro values handled.

```
df.isnull().sum()
```

	0
_num	0
id	0
rank	0
workers	0
company	0
url	0
state_l	0
state_s	0
city	0
metro	0
growth	0
revenue	0
industry	0
yrs_on_list	0

dtype: int64

```
print("Duplicate rows:", df.duplicated().sum())
```

Duplicate rows: 0

```
df.dtypes
```

	0
_num	int64
id	int64
rank	int64
workers	int64
company	object
url	object
state_l	object
state_s	object
city	object
metro	object
growth	float64
revenue	int64
industry	object
yrs_on_list	int64

dtype: object

```
print("Final dataset shape:", df.shape)
print("\nMissing values:")
print(df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
```

Final dataset shape: (5000, 14)

Missing values:

_num	0
id	0
rank	0
workers	0
company	0
url	0
state_l	0
state_s	0
city	0
metro	0
growth	0
revenue	0
industry	0
yrs_on_list	0

dtype: int64

Duplicate rows: 0

```
df[['rank', 'workers', 'growth', 'revenue', 'yrs_on_list']].describe()
```

	rank	workers	growth	revenue	yrs_on_list
count	5000.000000	5000.000000	5000.000000	5.000000e+03	5000.000000
mean	2500.500000	208.969800	516.439892	4.305818e+07	2.744400
std	1443.520003	1074.600125	2786.061791	1.818559e+08	1.999067
min	1.000000	0.000000	42.447000	1.953000e+06	1.000000
25%	1250.750000	24.000000	84.208250	4.876791e+06	1.000000
50%	2500.500000	50.000000	151.722800	1.072208e+07	2.000000
75%	3750.250000	125.000000	347.652825	2.695213e+07	4.000000
max	5000.000000	34219.000000	158956.910600	5.528203e+09	12.000000

```
industry_counts = df['industry'].value_counts()
```

```
industry_counts.head(10)
```

	count
industry	
IT Services	733
Advertising & Marketing	453
Business Products & Services	435
Health	377
Software	338
Financial Services	278
Consumer Products & Services	230
Manufacturing	212
Government Services	212
Retail	200

dtype: int64

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
print("Graph libraries ready")
```

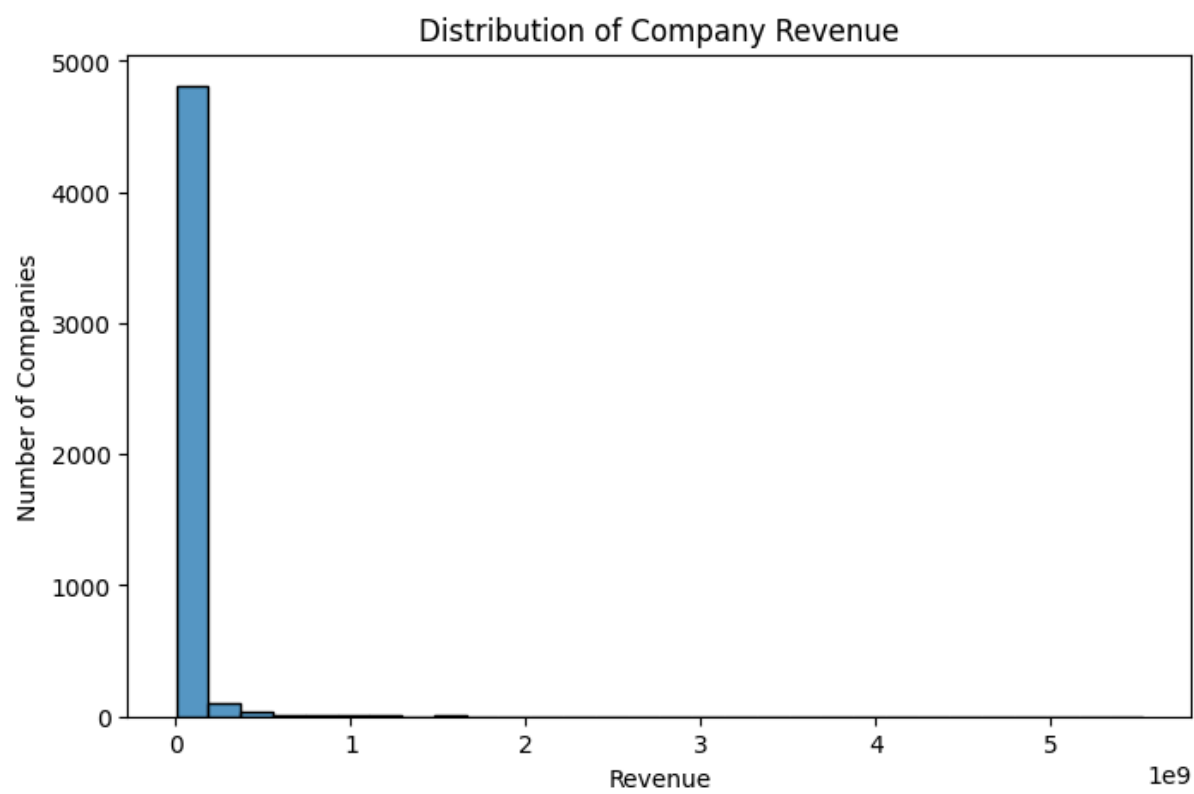
Graph libraries ready

```
plt.figure(figsize=(8,5))
```

```
sns.histplot(df['revenue'], bins=30)
```

```
plt.title('Distribution of Company Revenue')
plt.xlabel('Revenue')
plt.ylabel('Number of Companies')

plt.show()
```



```
industry_counts = df['industry'].value_counts()
industry_counts.head(10)
```

	count
industry	
IT Services	733
Advertising & Marketing	453
Business Products & Services	435
Health	377
Software	338
Financial Services	278
Consumer Products & Services	230
Manufacturing	212
Government Services	212
Retail	200

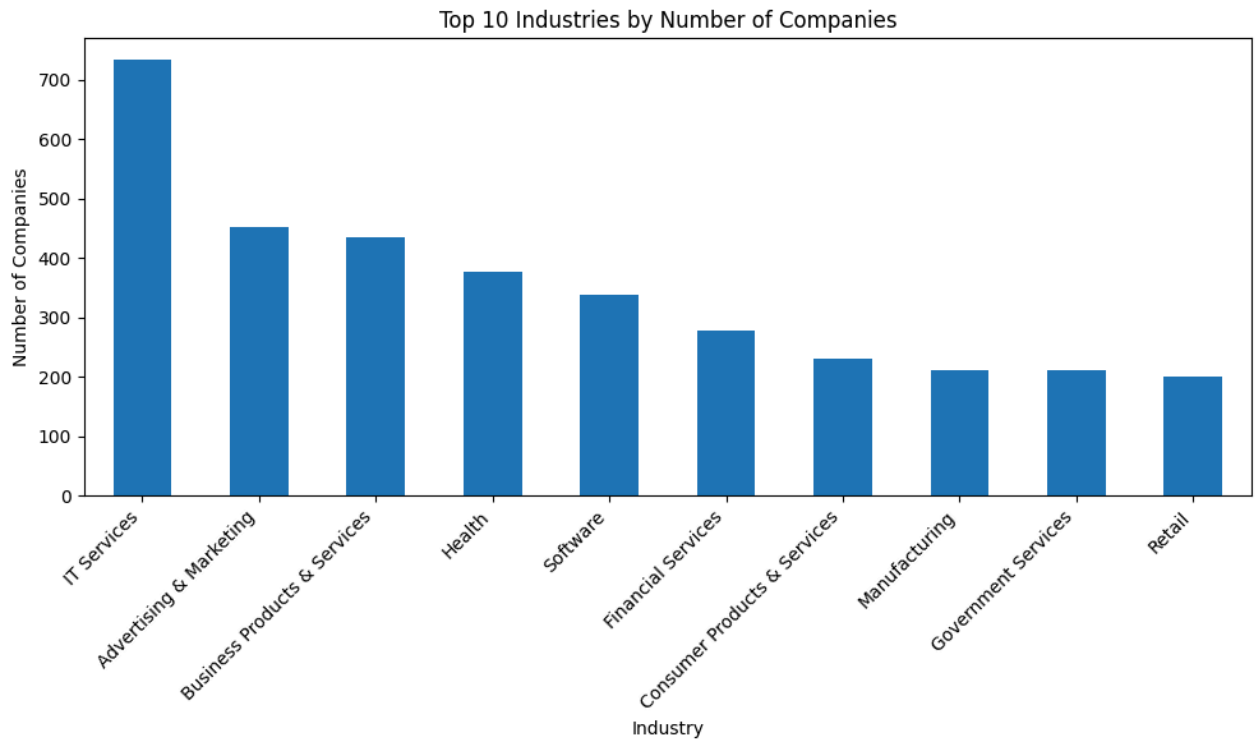
dtype: int64

```
plt.figure(figsize=(10,6))

industry_counts.head(10).plot(kind='bar')

plt.title('Top 10 Industries by Number of Companies')
plt.xlabel('Industry')
plt.ylabel('Number of Companies')
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```



```
print(df.columns.tolist())
```

```
['_num', 'id', 'rank', 'workers', 'company', 'url', 'state_l', 'state_s', 'city', 'metro']
```

```
state_counts = df['state_s'].value_counts()
```

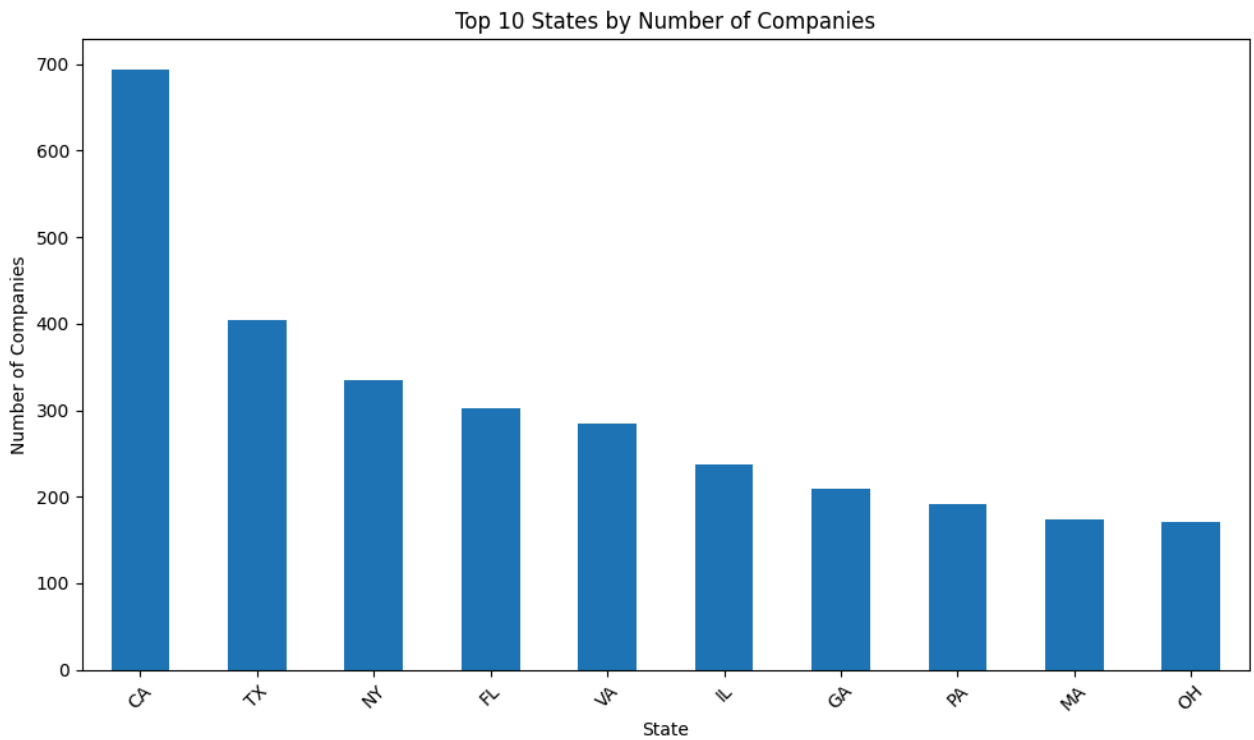
```
print(state_counts.head(10))
```

```
state_s
CA    694
TX    404
NY    335
FL    303
VA    284
IL    238
GA    209
PA    191
MA    174
OH    171
Name: count, dtype: int64
```

```
plt.figure(figsize=(10,6))
state_counts.head(10).plot(kind='bar')

plt.title('Top 10 States by Number of Companies')
plt.xlabel('State')
plt.ylabel('Number of Companies')

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

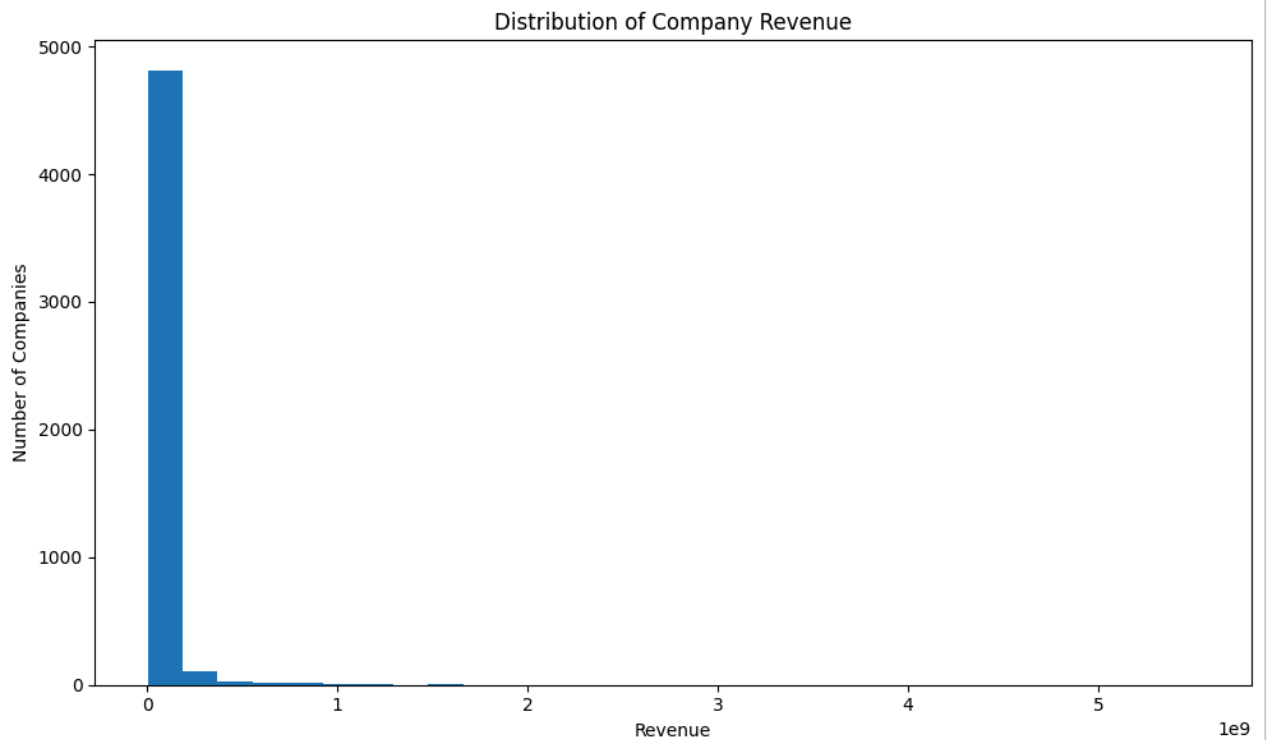


```
plt.figure(figsize=(10,6))

plt.hist(df['revenue'], bins=30)

plt.title('Distribution of Company Revenue')
plt.xlabel('Revenue')
plt.ylabel('Number of Companies')

plt.tight_layout()
plt.show()
```



```
top_revenue = df[['company', 'revenue']].sort_values(
    by='revenue',
    ascending=False
).head(10)
```

top_revenue

	company	revenue
2662	Pacific Union Real Estate	5528202691
4949	ABC Supply	4911166514
3690	Eco-Energy Global Biofuels	4138790000
1982	EnvisionRxOptions	3644869906
256	Bridger	3487872233
4618	Prime Therapeutics	2511716640
4203	Flyers Energy	1977921907
3425	Sun Coast Resources	1868964476
3240	Carahsoft Technology	1724988401
3820	TriNet	1644275000

```
plt.figure(figsize=(10,6))

top_revenue.sort_values('revenue').plot(
    x='company',
    y='revenue',
```

```

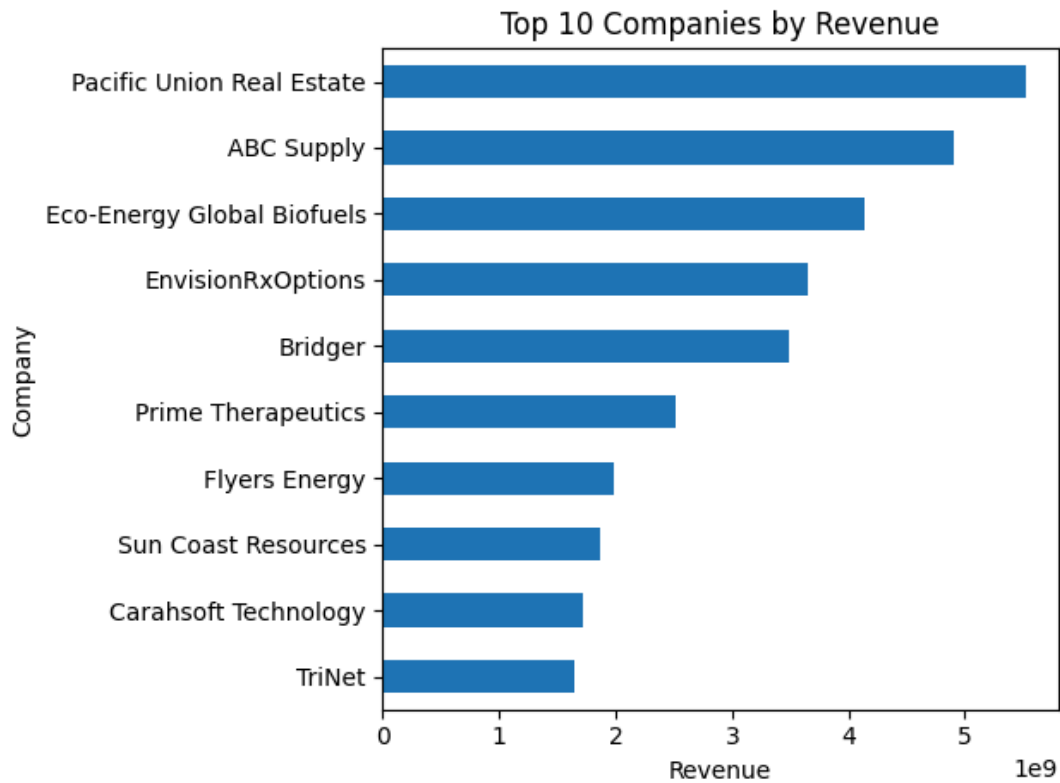
        kind='barh',
        legend=False
    )

    plt.title('Top 10 Companies by Revenue')
    plt.xlabel('Revenue')
    plt.ylabel('Company')

    plt.tight_layout()
    plt.show()

```

<Figure size 1000x600 with 0 Axes>



```

top_growth = df[['company', 'growth']].sort_values(
    by='growth',
    ascending=False
).head(10)
top_growth

```

	company	growth
0	Fuhu	158956.9106
1	Quest Nutrition	57347.9246
2	Reliant Asset Management	55460.1646
3	Superfish	26042.9630
4	Acacia Communications	20690.4578
5	Provider Power	19876.5169
6	Crescendo Bioscience	18375.0693
7	Plexus Worldwide	16457.6529
8	Vacasa	16192.0610
9	Go Energies	16006.8013

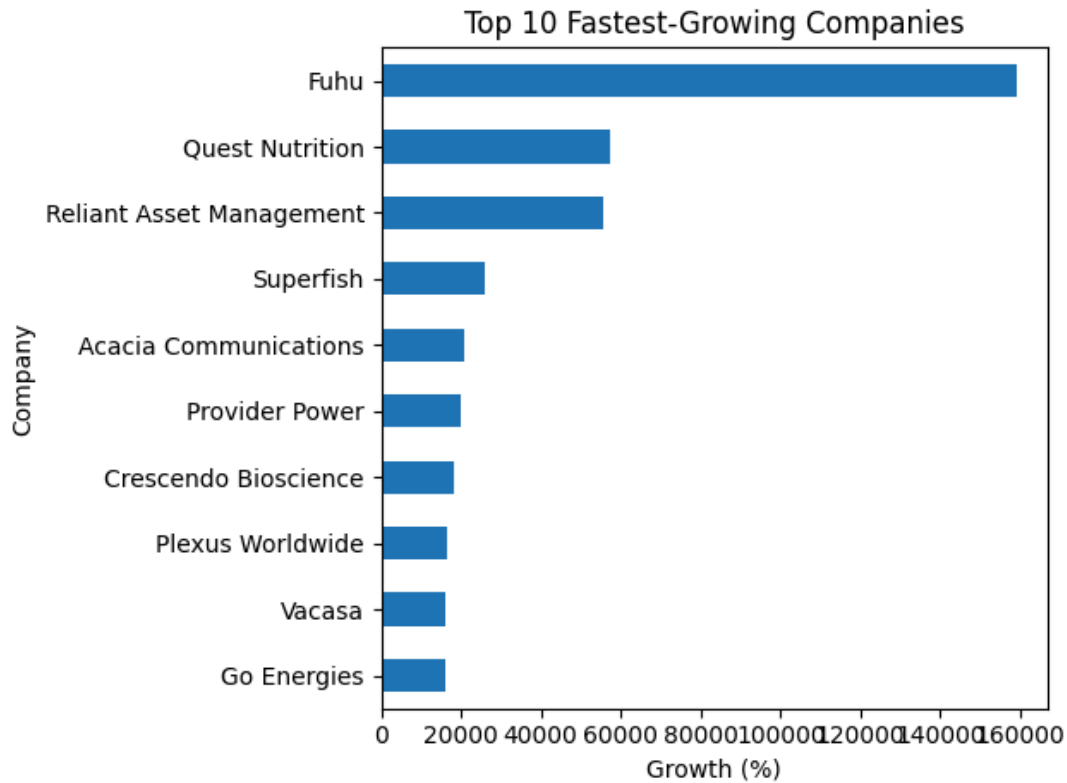
```
plt.figure(figsize=(10,6))

top_growth.sort_values('growth').plot(
    x='company',
    y='growth',
    kind='barh',
    legend=False
)

plt.title('Top 10 Fastest-Growing Companies')
plt.xlabel('Growth (%)')
plt.ylabel('Company')

plt.tight_layout()
plt.show()
```

<Figure size 1000x600 with 0 Axes>



```
numeric_data = df[
    ['rank', 'workers', 'growth', 'revenue', 'yrs_on_list']
]

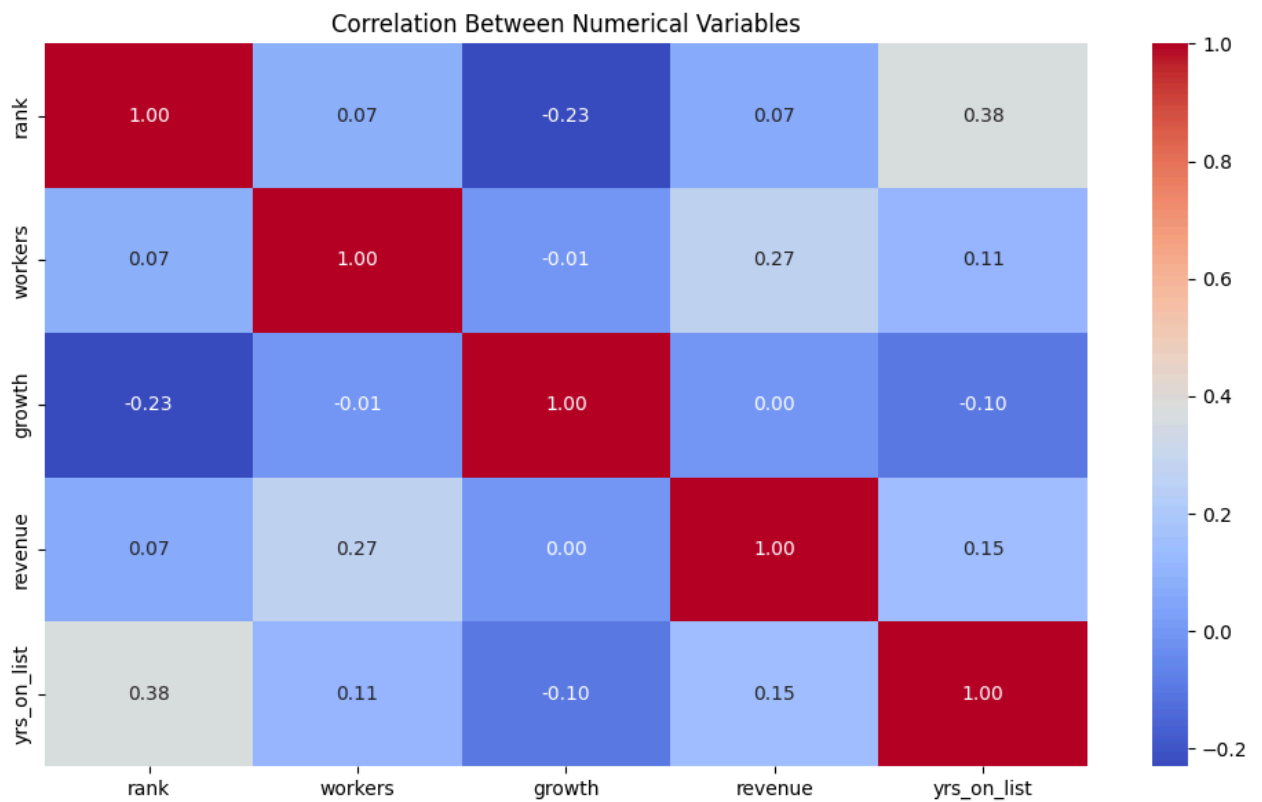
correlation = numeric_data.corr()
correlation
```

	rank	workers	growth	revenue	yrs_on_list
rank	1.000000	0.073496	-0.230007	0.067438	0.376546
workers	0.073496	1.000000	-0.012550	0.265699	0.110014
growth	-0.230007	-0.012550	1.000000	0.001715	-0.097157
revenue	0.067438	0.265699	0.001715	1.000000	0.150523
yrs_on_list	0.376546	0.110014	-0.097157	0.150523	1.000000

```
plt.figure(figsize=(10,6))

sns.heatmap(
    correlation,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Correlation Between Numerical Variables')
plt.tight_layout()
plt.show()
```



```
industry_revenue = (  
    df.groupby('industry')['revenue']  
      .sum()  
      .sort_values(ascending=False)  
      .head(10)  
      )  
industry_revenue
```

revenue	
industry	
Health	22022876049
Energy	21602659306
IT Services	19363587015
Business Products & Services	17175083779
Financial Services	12212242543
Human Resources	12174441948
Logistics & Transportation	11208388344
Consumer Products & Services	10746810431
Construction	10342377737
Telecommunications	9336973728

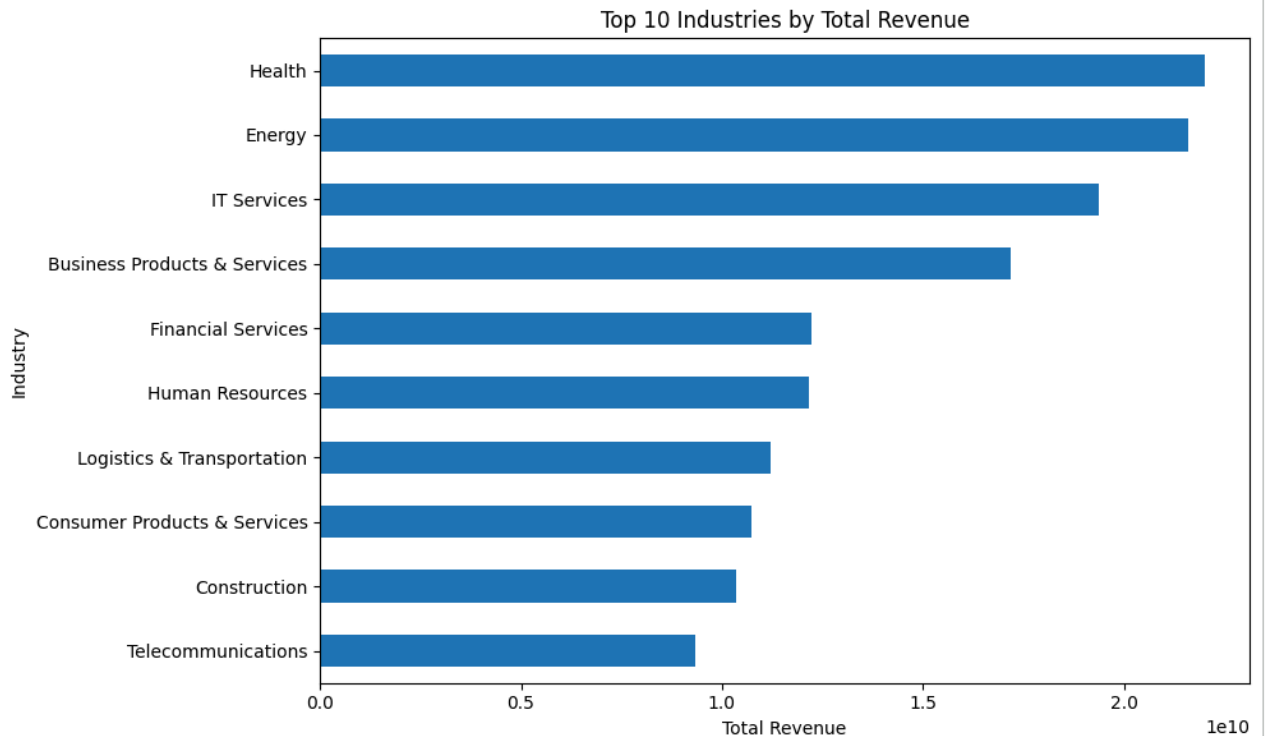
dtype: int64

```
plt.figure(figsize=(10,6))

industry_revenue.sort_values().plot(kind='barh')

plt.title('Top 10 Industries by Total Revenue')
plt.xlabel('Total Revenue')
plt.ylabel('Industry')

plt.tight_layout()
plt.show()
```



```
import os

print(os.listdir())

['.config', 'sample_data']
```

```
from google.colab import files

uploaded = files.upload()
```

[Choose files](#) Data Set- Inc...ist_2014.csv
Data Set- Inc5000 Company List_2014.csv(text/csv) - 1127413 bytes, last modified: 08/09/2026 - 100% done
 Saving Data Set- Inc5000 Company List_2014.csv to Data Set- Inc5000 Company List_2014.csv

```
import pandas as pd

df = pd.read_csv('Data Set- Inc5000 Company List_2014.csv')
```

```
# Remove unnecessary technical columns
columns_to_remove = [
```

```
        '_input',
        '_widgetName',
        '_source',
        '_resultNumber',
        '_pageUrl'
    ]

df = df.drop(columns=columns_to_remove, errors='ignore')

# Handle missing metro values
df['metro'] = df['metro'].fillna('Unknown')

# Remove duplicate rows
df = df.drop_duplicates()

# Save cleaned dataset
df.to_csv('cleaned_inc5000_data.csv', index=False)

print("✅ Cleaned CSV created successfully!")
print("Dataset shape:" df.shape)
```

✅ Cleaned CSV created successfully!
Dataset shape: (5000, 14)

```
from google.colab import files

files.download('cleaned_inc5000_data.csv')
```